

Postgres tables

```
external=# CREATE SCHEMA network;
TABLE network.plans (
  plan_id SMALLINT PRIMARY KEY,
  plan_name VARCHAR(50),
  price NUMERIC(10,2),
  internet_gb INTEGER,
  voice_minutes INTEGER
);CREATE SCHEMA
```

```
nada_mahmoud467@DESKTOP-F1F883M:~$ docker exec -it external_postgres_db bash
14094d109c13:/# psql -U external -d external
psql (15.1)
Type "help" for help.
```

```
external=#
external=# CREATE TABLE network.customers (
external(#   customer_id BIGINT PRIMARY KEY,
external(#   gender VARCHAR(10),
external(#   birth_date DATE,
external(#   city VARCHAR(50),
external(#   segment VARCHAR(20)
```

```
external=#
external=# CREATE TABLE network.subscriptions (subscription_id BIGINT PRIMARY KEY,
er_id external(# customer_id BIGINT,
external(# plan_id SMALLINT,
external(# msisdn VARCHAR(15),
external(# activation_date DATE,
external(# status VARCHAR(20));
CREATE TABLE
```

```
external=# CREATE TABLE network.cell_towers(cell_id VARCHAR(20) PRIMARY KEY,
external(# technology VARCHAR(10),
external(# city VARCHAR(50),
external(# latitude NUMERIC(9,6),
external(# longitude NUMERIC(9,6));
CREATE TABLE
```

```
external=# ALTER TABLE network.customers
```

```
external=# DROP COLUMN city;
```

```
ALTER TABLE
```

```
external=# \d network.customers
```

Table "network.customers"				
Column	Type	Collation	Nullable	Default
customer_id	bigint		not null	
gender	character varying(10)			
birth_date	date			
segment	character varying(20)			

```
Indexes:
```

```
"customers_pkey" PRIMARY KEY, btree (customer_id)
```

```
external=# \dt network.*
          List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
network | cell_towers    | table | external
network | customers      | table | external
network | plans          | table | external
network | subscriptions  | table | external
(4 rows)
```

```
external=# ALTER TABLE network.cell_towers drop column city;
```

```
external=# \d network.cell_towers;
          Table "network.cell_towers"
  Column      |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
cell_id       | character varying(20) |           | not null |
technology    | character varying(10) |           |          |
latitude      | numeric(9,6)       |           |          |
longitude     | numeric(9,6)       |           |          |
Indexes:
    "cell_towers_pkey" PRIMARY KEY, btree (cell_id)
```

```

external=#
external=# create table network.cities(city_id      SMALLINT PRIMARY KEY,
external(# city_name    VARCHAR(50),
external(# region       VARCHAR(30),
external(#
external(# latitude     NUMERIC(8,5),
external(# longitude    NUMERIC(8,5));
CREATE TABLE
external=# \dt network.*

```

List of relations			
Schema	Name	Type	Owner
network	cell_towers	table	external
network	cities	table	external
network	customers	table	external
network	plans	table	external
network	subscriptions	table	external

(5 rows)

```

external=# ALTER TABLE network.customers ADD COLUMN city_id      SMALLINT;
ALTER TABLE
external=# \d network.customers;

```

Table "network.customers"					
Column	Type	Collation	Nullable	Default	
customer_id	bigint		not null		
gender	character varying(10)				
birth_date	date				
segment	character varying(20)				
city_id	smallint				

Indexes:

"customers_pkey" PRIMARY KEY, btree (customer_id)

```

external=# ALTER TABLE network.cell_towers ADD COLUMN city_id      SMALLINT;
ALTER TABLE
external=# \d network.cell_towers;

```

Table "network.cell_towers"					
Column	Type	Collation	Nullable	Default	
cell_id	character varying(20)		not null		
technology	character varying(10)				
latitude	numeric(9,6)				
longitude	numeric(9,6)				
city_id	smallint				

Indexes:

"cell_towers_pkey" PRIMARY KEY, btree (cell_id)

Data Batch Generation

```
✓ telecom-network-analytics
> __pycache__
> data
✓ generators
  > __pycache__
  • __init__.py
  • config.py
  • generate_cities.py
  • generate_city_towers.py
  • generate_customers.py
  • generate_plans.py
  • generate_subscriptions.py
• db.py
• main.py
≡ requirements.txt
• test_connection.py
```

```
telecom-network-analytics > generators > • config.py > ...
1  DB_CONFIG = {
2      "host": "127.0.0.1",
3      "port": 5432,
4      "dbname": "external",
5      "user": "external",
6      "password": "external"
7  }
8
9  |
```

```

1  import pandas as pd
2  from sqlalchemy import create_engine
3  from pathlib import Path
4
5  from config import DB_CONFIG
6
7  # -----
8  # 1. Create Cities Data
9  # -----
10
11 cities_data = [
12     {
13         "city_id": 1,
14         "city_name": "Cairo",
15         "region": "Greater Cairo",
16         "latitude": 30.0444,
17         "longitude": 31.2357
18     },
19     {
20         "city_id": 2,
21         "city_name": "Giza",
22         "region": "Greater Cairo",
23         "latitude": 30.0131,
24         "longitude": 31.2089
25     },
26     {
27         "city_id": 3,
28         "city_name": "Alexandria",
29         "region": "North Coast",
30         "latitude": 31.2001,
31         "longitude": 29.9187
32     }
33 ]

```

```
32     },
33     {
34         "city_id": 4,
35         "city_name": "Mansoura",
36         "region": "Delta",
37         "latitude": 31.0409,
38         "longitude": 31.3785
39     },
40     {
41         "city_id": 5,
42         "city_name": "Tanta",
43         "region": "Delta",
44         "latitude": 30.7865,
45         "longitude": 31.0004
46     },
47     {
48         "city_id": 6,
49         "city_name": "Aswan",
50         "region": "Upper Egypt",
51         "latitude": 24.0889,
52         "longitude": 32.8998
53     },
54     {
55         "city_id": 7,
56         "city_name": "Luxor",
57         "region": "Upper Egypt",
58         "latitude": 25.6872,
59         "longitude": 32.6396
60     },
61     {
62         "city_id": 8,
63         "city_name": "Port Said",
```

```

69         "city_id": 9,
70         "city_name": "Suez",
71         "region": "Canal",
72         "latitude": 29.9668,
73         "longitude": 32.5498
74     },
75     {
76         "city_id": 10,
77         "city_name": "Sharm El Sheikh",
78         "region": "Sinai",
79         "latitude": 27.9158,
80         "longitude": 34.3300
81     }
82 ]
83
84
85 cities_df = pd.DataFrame(cities_data)
86

```

```

86
87
88 # -----
89 # 2. Save as Parquet
90 # -----
91
92 output_path = Path("../data/parquet/cities.parquet")
93
94 output_path.parent.mkdir(
95     parents=True,
96     exist_ok=True
97 )
98
99 cities_df.to_parquet(
100     output_path,
101     index=False
102 )
103
104 print("Cities parquet created successfully")
105
106

```

```

107 # -----
108 # 3. Load into PostgreSQL
109 # -----
110
111 connection_string = (
112     f"postgresql+psycopg://\"
113     f"{DB_CONFIG['user']}:"
114     f"{DB_CONFIG['password']}@"
115     f"{DB_CONFIG['host']}:"
116     f"{DB_CONFIG['port']}/\"
117     f"{DB_CONFIG['database']}\""
118 )
119
120 print(connection_string)
121
122 engine = create_engine(connection_string)
123
124
125 cities_df.to_sql(
126     "cities",
127     engine,
128     schema="network",
129     if_exists="append",
130     index=False
131 )
132
133
134 print("Cities loaded into PostgreSQL successfully")

```

- `(.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.generate_cities`
 Cities parquet created successfully
 postgresql+psycopg://external:external@127.0.0.1:5432/external
 Cities loaded into PostgreSQL successfully

```
1 from db import get_connection
```

```
2
```

```
3
```

```
4 plans = [
```

```
5
```

```
6     (
```

```
7         1,
```

```
8         "Basic",
```

```
9         "CONSUMER",
```

```
10        75,
```

```
11        6,
```

```
12        600
```

```
13    ),
```

```
14
```

```
15    (
```

```
16        2,
```

```
17        "Medium",
```

```
18        "CONSUMER",
```

```
19        150,
```

```
20        18,
```

```
21        1600
```

```
22    ),
```

```
23
```

```
24    (
```

```
25        3,
```

```
26        "Premium",
```

```
27        "CONSUMER",
```

```
28        400,
```

```
29        55,
```

```
30        4000
```

```
31    ),
```

```
32
```

```
34         4,  
35         "Corporate VIP",  
36         "ENTERPRISE",  
37         850,  
38         140,  
39         10000  
40     )  
41 ]  
42  
43  
44 def generate_plans():  
45  
46     conn = get_connection()  
47     cur = conn.cursor()  
48  
49  
50     cur.executemany(  
51         """  
52         INSERT INTO network.plans  
53         (  
54             plan_id,  
55             plan_name,  
56             plan_type,  
57             price,  
58             internet_gb,  
59             voice_minutes  
60         )  
61         VALUES  
62         (%s,%s,%s,%s,%s,%s)  
63  
64         ON CONFLICT(plan_id)  
65         DO NOTHING
```

```

64         ON CONFLICT(plan_id)
65         DO NOTHING
66         """
67         plans
68     )
69
70
71     conn.commit()
72
73     cur.close()
74     conn.close()
75
76     print("Plans generated successfully")
77
78
79     if __name__ == "__main__":
80         generate_plans()

```

```

try:
    # For help:
    external=# \q
    (.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.generate_plans
    Plans generated successfully
    (.venv) PS F:\Airflow\telecom-network-analytics> docker exec -it external_postgres_db psql -U external -d external
    psql (15.1)

    external=# SELECT * FROM network.plans;
    plan_id | plan_name | price | internet_gb | voice_minutes | plan_type
    -----+-----+-----+-----+-----+-----
    1 | Basic | 75.00 | 6 | 600 | CONSUMER
    2 | Medium | 150.00 | 18 | 1600 | CONSUMER
    3 | Premium | 400.00 | 55 | 4000 | CONSUMER
    4 | Corporate VIP | 850.00 | 140 | 10000 | ENTERPRISE
    (4 rows)

```

telecom-network-analytics > generators > generate_customers.py > generate_birth_dates

```
1  import numpy as np
2  import pandas as pd
3  from faker import Faker
4  from db import get_connection
5
6
7  fake = Faker()
8
9
10 CUSTOMERS = 1_000_000
11
12
13 city_distribution = {
14     1:0.30,    # Cairo
15     2:0.15,    # Giza
16     3:0.10,    # Alexandria
17     4:0.05,    # New Capital
18     5:0.10,
19     6:0.20,
20     7:0.05,
21     8:0.05
22 }
23
24
25
26
27 from datetime import date
28
```

```
29
30 def generate_birth_dates(size):
31
32     current_year = date.today().year
33
34     ages = np.random.choice(
35         [
36             22,
37             35,
38             50,
39             65
40         ],
41         size=size,
42         p=[
43             0.25,
44             0.40,
45             0.25,
46             0.10
47         ]
48     )
49
50
51     birthdays = []
52
53     for age in ages:
54
55         birth_year = current_year - int(age)
```

```
51 birthdays = []
52
53 for age in ages:
54     birth_year = current_year - int(age)
55
56     birth_date = fake.date_between_dates(
57         date_start=date(
58             birth_year,
59             1,
60             1
61         ),
62         date_end=date(
63             birth_year,
64             12,
65             31
66         )
67     )
68
69     birthdays.append(birth_date)
70
71
72
73 return birthdays
74
```

```
76
77 def generate_customers():
78
79
80     conn=get_connection()
81     cur=conn.cursor()
82
83
84     batch=[]
85
86
87     city_ids=list(city_distribution.keys())
88     city_prob=list(city_distribution.values())
89
90
91     cities=np.random.choice(
92         city_ids,
93         CUSTOMERS,
94         p=city_prob
95     )
96
97
98     segments=np.random.choice(
99         [
100             "Individual",
101             "Business"
```

```
97
98     segments=np.random.choice(
99         [
100             "Individual",
101             "Business"
102         ],
103         CUSTOMERS,
104         p=[
105             0.92,
106             0.08
107         ]
108     )
109
110
111     genders=np.random.choice(
112         [
113             "Male",
114             "Female"
115         ],
116         CUSTOMERS,
117         p=[
118             0.52,
119             0.48
120         ]
121     )
122
```

```
123
124     birthdays=generate_birth_dates(
125         CUSTOMERS
126     )
127
128
129
130     for i in range(CUSTOMERS):
131
132         batch.append(
133             (
134                 i+1,
135                 genders[i],
136                 birthdays[i],
137                 segments[i],
138                 int(cities[i])
139             )
140         )
141
142
143         if len(batch)==10000:
144
145             cur.executemany(
146                 """
147                 INSERT INTO network.customers
148                 (
149
```

```
146         cur.executemany(
147             """
148             INSERT INTO network.customers
149             (
150             customer_id,
151             gender,
152             birth_date,
153             segment,
154             city_id
155             )
156             VALUES
157             (%s,%s,%s,%s,%s)
158             """,
159             batch
160         )
161
162         conn.commit()
163         batch=[]
164
165         print(i,"customers inserted")
166
167
168     conn.commit()
169
170     cur.close()
171     conn.close()
172
```

```
171     conn.close()
172
173
174     print("Customers generation completed")
175
176
177
178 if __name__=="__main__":
179     generate_customers()
```

```
external=# \q
• (.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.generate_customers
9999 customers inserted
19999 customers inserted
29999 customers inserted
```

```
external=# SELECT COUNT(*) FROM network.customers;
count
-----
1000000
(1 row)
```

```
telecom-network-analytics > generators > generate_subscriptions.py > ...
```

```
1  import random
2  import numpy as np
3  from datetime import date, timedelta
4
5  from db import get_connection
6
7
8  TOTAL_SUBSCRIPTIONS = 1_200_000
9  TOTAL_CUSTOMERS = 1_000_000
10
11
12  def generate_msisdn():
13
14      prefix = np.random.choice(
15          [
16              "010",
17              "011",
18              "012",
19              "015"
20          ]
21      )
22
23      number = random.randint(
24          10000000,
25          99999999
26      )
27
28      return f"+20{prefix}{number}"
29
```

```
30
31
32 def generate_subscription_type():
33
34     return np.random.choice(
35         [
36             "CONSUMER",
37             "ENTERPRISE",
38             "IOT"
39         ],
40         p=[
41             0.80,
42             0.18,
43             0.02
44         ]
45     )
```

```
48
49 def choose_plan(subscription_type):
50
51     if subscription_type == "ENTERPRISE":
52
53         return 4
54
55
56     elif subscription_type == "IOT":
57
58         # IoT uses low plans
59         return np.random.choice(
60             [
61                 1,
62                 2
63             ],
64             p=[
65                 0.8,
66                 0.2
67             ]
68         )
69
70
71     else:
72
```

```
71     else:
72
73         return np.random.choice(
74             [
75                 1,
76                 2,
77                 3
78             ],
79             p=[
80                 0.40,
81                 0.45,
82                 0.15
83             ]
84         )
85
```

```
87
88 def generate_status():
89
90     return np.random.choice(
91         [
92             "Active",
93             "Suspended",
94             "Terminated"
95         ],
96         p=[
97             0.88,
98             0.09,
99             0.03
100         ]
101     )
102
```

```
104
105 def generate_subscriptions():
106
107     conn = get_connection()
108     cur = conn.cursor()
109
110
111     batch=[]
112
113
114     for i in range(1, TOTAL_SUBSCRIPTIONS+1):
115
116
117         customer_id = random.randint(
118             1,
119             TOTAL_CUSTOMERS
120         )
121
122
123         sub_type = generate_subscription_type()
124
125
126         plan_id = choose_plan(
127             sub_type
128         )
129
130
131         activation_date = (
132             date.today()
133             -
134             timedelta(
135                 days=random.randint(
136                     30,
```

```

134         timedelta(
135             days=random.randint(
136                 30,
137                 2000
138             )
139         )
140     )
141
142     batch.append(
143         (
144             i,
145             customer_id,
146             plan_id,
147             generate_msisdn(),
148             activation_date,
149             generate_status(),
150             sub_type
151         )
152     )
153
154
155
156     if len(batch) == 10000:
157
158         cur.executemany(
159             """
160             INSERT INTO network.subscriptions
161             (

```

```
158
159     cur.executemany(
160         """
161         INSERT INTO network.subscriptions
162         (
163             subscription_id,
164             customer_id,
165             plan_id,
166             msisdn,
167             activation_date,
168             status,
169             subscription_type
170         )
171
172         VALUES
173         (
174             %s,%s,%s,%s,%s,%s,%s,%s
175         )
176         """,
177         batch
178     )
179
180
181     conn.commit()
182
183     batch.clear()
184
185     print(
186         i,
187         "subscriptions inserted"
188     )
189
```

```

190
191     conn.commit()
192
193     cur.close()
194     conn.close()
195
196
197     print(
198         "Subscriptions generation completed"
199     )
200
201
202
203 if __name__ == "__main__":
204     generate_subscriptions()

```

```

• (.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.generate_subscriptions
10000 subscriptions inserted
20000 subscriptions inserted
30000 subscriptions inserted
40000 subscriptions inserted
50000 subscriptions inserted

```

```

external=# Select count(*) from network.subscriptions;
count
-----
1200000
1 row)

```

telecom-network-analytics > generators > generate_city_towers.py > generate_cell_towers

```
1  import psycpg
2  import random
3
4  from generators.config import DB_CONFIG
5
6
7  TOWERS_COUNT = 1500
8
9
10 def generate_cell_towers():
11
12     conn = psycpg.connect(**DB_CONFIG)
13     cur = conn.cursor()
14
15
16     # Load cities coordinates
17     cur.execute("""
18         SELECT
19             city_id,
20             latitude,
21             longitude,
22             city_name
23         FROM network.cities
24     """)
25
26
27     cities = cur.fetchall()
28
```

```
29
30 # city weights based on telecom density
31 city_weights = []
32
33 for city in cities:
34
35     city_id = city[0]
36
37     if city_id == 1:      # Cairo
38         | weight = 30
39
40     elif city_id == 2:    # Giza
41         | weight = 20
42
43     elif city_id == 3:    # Alexandria
44         | weight = 15
45
46     elif city_id in [4,5]:
47         | weight = 8
48
49     else:
50         | weight = 2
51
52
53     city_weights.append(weight)
54
55
```

```
56
57     batch=[]
58
59
60     for i in range(1,TOWERS_COUNT+1):
61
62         city=random.choices(
63             cities,
64             weights=city_weights
65         )[0]
66
67
68         city_id = city[0]
69         base_lat = float(city[1])
70         base_lon = float(city[2])
71
72
73         # Technology distribution
74
75         technology=random.choices(
76             ["4G","5G"],
77             weights=[85,15]
78         )[0]
79
```

```
80
81     # 5G concentrated in major cities
82
83     if technology=="5G":
84         |
85         |     if city_id not in [1,2,3]:
86         |         |     technology="4G"
87         |
88
89
90     # tower coordinates around city
91
92     latitude = round(
93         |     base_lat + random.uniform(-0.08,0.08),
94         |     6
95     )
96
97
98     longitude = round(
99         |     base_lon + random.uniform(-0.08,0.08),
100        |     6
101    )
```

```
105     batch.append(  
106         (  
107             f"CELL_{i:06d}",  
108             technology,  
109             latitude,  
110             longitude,  
111             city_id  
112         )  
113     )  
114  
115  
116     if len(batch)==500:  
117  
118  
119         cur.executemany(  
120             """  
121             INSERT INTO network.cell_towers  
122             (  
123                 cell_id,  
124                 technology,  
125                 latitude,  
126                 longitude,  
127                 city_id  
128             )  
129             VALUES  
130             (%s,%s,%s,%s,%s)  
131             """,  
132             batch
```

```

129         VALUES
130         (%s,%s,%s,%s,%s)
131         """,
132         batch
133     )
134
135
136     conn.commit()
137
138     print(
139         f"{i} towers inserted"
140     )
141
142     batch=[]
143
144
145
146     if batch:
147
148         cur.executemany(
149             """
150             INSERT INTO network.cell_towers
151             VALUES (%s,%s,%s,%s,%s)
152             """,
153             batch
154         )
155
156         conn.commit()

```

```

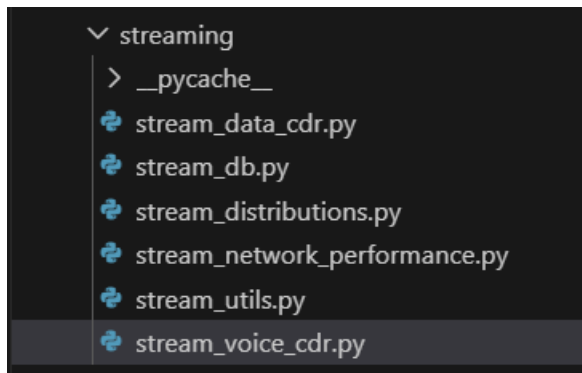
158
159
160     cur.close()
161     conn.close()
162
163
164     print("Cell towers generation completed")
165
166
167
168     if __name__=="__main__":
169         generate_cell_towers()

```

```
(.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.generate_city_towers
500 towers inserted
1000 towers inserted
1500 towers inserted
Cell towers generation completed
```

```
external=# select count(*) from network.cell_towers;
count
-----
1500
(1 row)
```

Streaming Generation



Airflow > telecom-network-analytics > generators > streaming >  stream_voice_cdr.py > ...

```
1  import json
2  import random
3  import time
4
5  from datetime import datetime, timedelta
6
7  from kafka import KafkaProducer
8
9
10 from generators.streaming.stream_db import (
11     load_subscriptions,
12     load_towers
13 )
14
15
16 from generators.streaming.stream_distributions import (
17     weighted_choice,
18     VOICE_STATUS,
19     TERMINATION
20 )
21
22
23 from generators.streaming.stream_utils import (
24     generate_call_duration
25 )
26
27
28
29 KAFKA_SERVER = "localhost:9092"
30
31 TOPIC_NAME = "telecom_voice_cdr"
32
```

```
30
31 TOPIC_NAME = "telecom_voice_cdr"
32
33
34 EVENTS_PER_SECOND = 50
35
36
37
38
39 producer = KafkaProducer(
40     bootstrap_servers=KAFKA_SERVER,
41     value_serializer=lambda x:
42         json.dumps(x).encode("utf-8")
43 )
44
45
46
47
48
49
50 print(
51     "Connected:",
52     producer.bootstrap_connected()
53 )
```

```
57
58 # -----
59 # Generate Voice CDR
60 # -----
61
62 def generate_voice_cdr(
63     cdr_id,
64     subscriptions,
65     towers
66 ):
67
68
69     subscription = random.choice(
70         subscriptions
71     )
72
73
74     subscription_id = subscription[0]
75
76
77
78     tower = random.choice(
79         towers
80     )
81
82
83     cell_id = tower["cell_id"]
84
85     technology = tower["technology"]
86
```

```
86
87
88
89     start_time = (
90         datetime.now()
91         -
92         timedelta(
93             seconds=random.randint(
94                 0,
95                 3600
96             )
97         )
98     )
99
100
101
102 )
103
104
105
106 call_status = weighted_choice(
107     VOICE_STATUS
108 )
109
110
111
112 termination = weighted_choice(
113     TERMINATION
114 )
115
116
```

```
117
118     # keep status and termination realistic
119
120     if call_status == "Completed":
121         termination = "NORMAL"
122
123
124
125     elif call_status == "Dropped":
126         termination = "DROP"
127
128
129
130
131     else:
132         termination = "NETWORK_FAILURE"
133
134
135
136
137
138     return {
139
140
141         "cdr_id":
142             cdr_id,
143
```

```
138     return {
139
140
141         "cdr_id":
142         |     cdr_id,
143
144
145
146         "subscription_id":
147         |     subscription_id,
148
149
150
151         "start_time":
152         |     start_time.strftime(
153         |         "%Y-%m-%d %H:%M:%S"
154         |     ),
155
156
157
158         "duration_seconds":
159         |     generate_call_duration(),
160
161
162
163         "cell_id":
164         |     cell_id,
165
```

```

166
167
168     "technology":
169         technology,
170
171
172
173     "call_status":
174         call_status,
175
176
177
178     "termination_cause":
179         termination,
180
181
182
183     "event_time":
184         datetime.now().strftime(
185             "%Y-%m-%d %H:%M:%S"
186         )
187
188 }
189

```

```
(.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.streaming.stream_voice_cdr
```

```
Connected: True
```

```
Loading subscriptions...
```

```
5 subscriptions loaded
```

```
Loading towers...
```

```
1500 towers loaded
```

```
Voice CDR streaming started...
```

```
{'cdr_id': 1, 'subscription_id': 5, 'start_time': '2026-07-15 12:13:16', 'duration_seconds': 70, 'cell_id': 'CELL_001387', 'technology': '4G', 'call_status': 'Completed', 'termination_cause': 'NORMAL', 'event_time': '2026-07-15 13:04:19'}
```

```
RecordMetadata(topic='telecom_voice_cdr', partition=2, topic_partition=TopicPartition(topic='telecom_voice_cdr', partition=2), offset=1211, timestamp=1784109859223, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=238, serialized_header_size=-1)
```

```
{'cdr_id': 2, 'subscription_id': 1, 'start_time': '2026-07-15 12:54:29', 'duration_seconds': 281, 'cell_id': 'CELL_000870', 'technology': '4G', 'call_status': 'Completed', 'termination_cause': 'NORMAL', 'event_time': '2026-07-15 13:04:19'}
```

```
RecordMetadata(topic='telecom_voice_cdr', partition=1, topic_partition=TopicPartition(topic='telecom_voice_cdr', partition=1), offset=1287, timestamp=1784109859256, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=239, serialized_header_size=-1)
```

```
(.venv) PS F:\Airflow\telecom-network-analytics> docker exec -it kafka-broker bash
> --bootstrap-server localhost:9092 \
> --topic telecom_voice_cdr \
> --from-beginning \
> --max-messages 5
{"cdr_id": 2842894408, "subscription_id": 3, "start_time": "2026-07-15 11:19:19", "duration_seconds": 96, "cell_id": "CELL_000611",
, "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-07-15 11:36:11"}
{"cdr_id": 8348937760, "subscription_id": 2, "start_time": "2026-07-15 11:09:31", "duration_seconds": 61, "cell_id": "CELL_000668",
, "technology": "5G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-07-15 11:36:11"}
{"cdr_id": 4619219969, "subscription_id": 3, "start_time": "2026-07-15 11:01:20", "duration_seconds": 81, "cell_id": "CELL_001287",
, "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-07-15 11:36:11"}
{"cdr_id": 9894181050, "subscription_id": 1, "start_time": "2026-07-15 11:32:06", "duration_seconds": 70, "cell_id": "CELL_001375",
, "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-07-15 11:36:11"}
{"cdr_id": 8159046806, "subscription_id": 4, "start_time": "2026-07-15 11:09:33", "duration_seconds": 78, "cell_id": "CELL_000517",
, "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-07-15 11:36:11"}
Processed a total of 5 messages
root@kafka-broker:/# ^C
root@kafka-broker:/# exit
exit
```

Airflow > telecom-network-analytics > generators > streaming >  stream_distributions.py > ...

```
1  import random
2
3
4  # -----
5  # Voice Call
6  # -----
7
8  VOICE_STATUS = [
9      ("Completed", 0.965),
10     ("Dropped", 0.015),
11     ("Failed", 0.020)
12 ]
13
14
15  TERMINATION = [
16     ("NORMAL", 0.85),
17     ("DROP", 0.10),
18     ("NETWORK_FAILURE", 0.05)
19 ]
20
21
22  # -----
23  # Technology
24  # -----
25
26  TECH_DISTRIBUTION = [
27     ("4G", 0.85),
28     ("5G", 0.15)
29 ]
30
```

```
30
31
32 # -----
33 # Peak Hours
34 # -----
35
36 PEAK_START = 19
37 PEAK_END = 23
38
39
40 # -----
41 # Utility
42 # -----
43
44 def weighted_choice(items):
45     values = [x[0] for x in items]
46
47     weights = [x[1] for x in items]
48
49
50     return random.choices(values,weights=weights,k=1)[0]
```

Airflow > telecom-network-analytics > generators > streaming >  stream_utils.py >  generate_call_duration

```
1  import random
2
3  from datetime import datetime
4
5
6  def is_peak_hour():
7
8      hour = datetime.now().hour
9
10     return 19 <= hour <= 23
11
12
13     # -----
14     # Voice Duration
15     # -----
16
17 def generate_call_duration():
18
19     r = random.random()
20
21     if r < 0.70:
22         return random.randint(60,120)
23
24     if r < 0.95:
25         return random.randint(120,300)
26
27     return random.randint(300,1800)
28
29
```

```
30 # -----
31 # 4G Throughput
32 # -----
33
34 def throughput_4g():
35     if is_peak_hour():
36         return round(random.uniform(15,28),2)
37     return round(random.uniform(25,45),2)
38
39
40
41
42
43 # -----
44 # 5G Throughput
45 # -----
46
47 def throughput_5g():
48     if is_peak_hour():
49         return round(random.uniform(120,250),2)
50     return round(random.uniform(180,450),2)
51
52
53
54
```

```
55
56 # -----
57 # Upload
58 # -----
59
60 def upload_speed(technology):
61
62     if technology=="4G":
63         return round(random.uniform(5,15),2)
64
65     return round(random.uniform(30,80),2)
66
67
68 # -----
69 # Latency
70 # -----
71
72 def latency(technology):
73
74     if technology=="4G":
75
76         if is_peak_hour():
77             return round(random.uniform(45,70),2)
78
79         return round(random.uniform(30,55),2)
80
81     return round(random.uniform(8,20),2)
82
```

```
84 # -----
85 # Packet Loss
86 # -----
87
88 def packet_loss():
89     if is_peak_hour():
90         return round(random.uniform(0.4,1.3),3)
91     return round(random.uniform(0.1,0.7),3)
92
93
94
95
96
97 # -----
98 # Availability
99 # -----
100
101 def availability():
102     if random.random()<0.01:
103         return round(random.uniform(90,98),2)
104     return round(random.uniform(99.5,100),2)
```

Airflow > telecom-network-analytics > generators > streaming >  stream_db.py >  get_cells

```
1  from psycopg2 import connect
2  from generators.config import POSTGRES_CONFIG
3
4
5  def get_cells():
6      conn = connect(**POSTGRES_CONFIG)
7      cur = conn.cursor()
8
9      cur.execute("""
10         SELECT cell_id, technology
11         FROM network.cell_towers
12     """)
13
14     rows = cur.fetchall()
15
16     cur.close()
17     conn.close()
18
19     return [
20         {
21             "cell_id": row[0],
22             "technology": row[1]
23         }
24         for row in rows
25     ]
26
```

```

27
28 def get_subscriptions():
29     conn = connect(**POSTGRES_CONFIG)
30     cur = conn.cursor()
31
32     cur.execute("""
33         SELECT *
34         FROM network.subscriptions
35         LIMIT 5
36     """)
37
38     rows = cur.fetchall()
39
40     cur.close()
41     conn.close()
42
43     return rows
44
45 load_subscriptions = get_subscriptions
46 load_towers = get_cells

```

```

8
9
10 POSTGRES_CONFIG = {
11     "host": "localhost",
12     "port": 5432,
13     "database": "external",
14     "user": "external",
15     "password": "external"
16 }
17
18
19 KAFKA_CONFIG = {
20     'bootstrap_servers': 'localhost:29092'
21 }

```

Airflow > telecom-network-analytics > generators > streaming >  stream_data_cdr.py > ...

```
1  import json
2  import time
3  import random
4
5  from datetime import datetime, timedelta
6
7  from kafka import KafkaProducer
8
9
10 from generators.streaming.stream_db import (
11     load_subscriptions,
12     load_towers
13 )
14
15
16 from generators.streaming.stream_distributions import (
17     weighted_choice
18 )
19
20
21 from generators.streaming.stream_utils import (
22     is_peak_hour
23 )
24
25
26
27 TOPIC = "telecom_cdr_data"
28
```

```
30
31 # -----
32 # Traffic Distribution
33 # -----
34
35 TRAFFIC_TYPES = [
36     ("Internet", 0.70),
37     ("Video", 0.15),
38     ("Voice", 0.10),
39     ("SMS", 0.05)
40 ]
41
42
43
44
45
```

```
30
31 # -----
32 # Traffic Distribution
33 # -----
34
35 TRAFFIC_TYPES = [
36     ("Internet", 0.70),
37     ("Video", 0.15),
38     ("Voice", 0.10),
39     ("SMS", 0.05)
40 ]
41
42
43
44
45
```

```
50 # -----
51 # Duration Generator
52 # -----
53
54 def generate_duration(traffic_type):
55
56
57     if traffic_type == "Internet":
58
59         return random.randint(
60             60,
61             3600
62         )
63
64
65     if traffic_type == "Video":
66
67         return random.randint(
68             300,
69             7200
70         )
71
72
73     if traffic_type == "Voice":
74
75         return random.randint(
76             30,
77             1800
78         )
```

```
72
73     if traffic_type == "Voice":
74
75         return random.randint(
76             30,
77             1800
78         )
79
80
81     return random.randint(
82         1,
83         5
84     )
85
86
87
88
89 # -----
90 # Data Volume Generator
91 # -----
92
93 def generate_data_volume(traffic_type):
94
95
96     peak = is_peak_hour()
```

```
93 def generate_data_volume(traffic_type):
94
95
96     peak = is_peak_hour()
97
98
99
100     if traffic_type == "Internet":
101
102
103         if peak:
104
105             return round(
106                 random.uniform(500,7000),
107                 2
108             )
109
110
111             return round(
112                 random.uniform(100,4000),
113                 2
114             )
115
116
117
118     if traffic_type == "Video":
```

```
117
118     if traffic_type == "Video":
119
120
121         if peak:
122
123             return round(
124                 random.uniform(1000,9000),
125                 2
126             )
127
128
129             return round(
130                 random.uniform(500,7000),
131                 2
132             )
133
134
135
136     if traffic_type == "Voice":
137
138
139         return round(
140             random.uniform(1,50),
141             2
142         )
143
```

```
145
146     return round(
147         random.uniform(0.001,1),
148         3
149     )
150
151
152
153
154
155     # -----
156     # CDR Generator
157     # -----
158
159     def generate_cdr(
160         cdr_id,
161         subscriptions,
162         towers
163     ):
164
165
166         traffic = weighted_choice(
167             TRAFFIC_TYPES
168         )
169
```

```
171
172     subscription = random.choice(
173         |
174         subscriptions
175     )
176
177     subscription_id = subscription[0]
178
179
180
181     tower = random.choice(
182         |
183         towers
184     )
185
186     cell_id = tower["cell_id"]
187
188     technology = tower["technology"]
189
190
191
192
193     start_time = (
194         |
195         datetime.now()
196     )
```

```
198
199         timedelta(
200             seconds=random.randint(
201                 0,
202                 86400
203             )
204         )
205
206     )
207
208
209
210     return {
211
212         "cdr_id":
213         |     cdr_id,
214
215
216
217
218         "subscription_id":
219         |     subscription_id,
220
221
```

```
216
217
218     "subscription_id":
219     |     subscription_id,
220
221
222
223     "cell_id":
224     |     cell_id,
225
226
227
228     "technology":
229     |     technology,
230
231
232
233     "traffic_type":
234     |     traffic,
235
236
237
238     "start_time":
239     |     start_time.strftime(
240     |         "%Y-%m-%d %H:%M:%S"
241     |     ),
242
```

```
244
245     "duration_seconds":
246         generate_duration(
247             traffic
248         ),
249
250
251
252     "data_volume_mb":
253         generate_data_volume(
254             traffic
255         ),
256
257
258
259     "event_time":
260         datetime.now().strftime(
261             "%Y-%m-%d %H:%M:%S"
262         )
263
264 }
```

```
269
270 # -----
271 # Kafka Streaming
272 # -----
273
274 def main():
275
276     producer = KafkaProducer(
277
278         bootstrap_servers="localhost:9092",
279
280         value_serializer=lambda x:
281             json.dumps(x).encode("utf-8")
282     )
283
284
285
286
287
288     print(
289         "Loading subscriptions..."
290     )
291
292
293     subscriptions = load_subscriptions()
294
295
```

```
print(  
    f"{len(subscriptions)} subscriptions loaded"  
)
```

```
print(  
    "Loading towers..."  
)
```

```
towers = load_towers()
```

```
print(  
    f"{len(towers)} towers loaded"  
)
```

```
print(  
    "CDR data streaming started"  
)
```

```
323
324     counter = 0
325
326
327
328     while True:
329
330
331         counter += 1
332
333
334
335         record = generate_cdr(
336
337             counter,
338
339             subscriptions,
340
341             towers
342
343         )
344
```

```
346
347     future = producer.send(
348         TOPIC,
349         record
350     )
351
352
353
354
355
356
357     try:
358
359         metadata = future.get(
360             timeout=10
361         )
362
363
364
365
366         print(record)
367
368         print(metadata)
369
370
```

```

373         except Exception as e:
374
375
376             print(
377                 "Kafka Error:",
378                 e
379             )
380
381
382
383             time.sleep(
384                 0.1
385             )
386
387
388
389
390
391 if __name__ == "__main__":
392
393     main()

```

KeyboardInterrupt

• (.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.streaming.stream_data_cdr

Loading subscriptions...

5 subscriptions loaded

Loading towers...

1500 towers loaded

CDR data streaming started

{'cdr_id': 1, 'subscription_id': 3, 'cell_id': 'CELL_001105', 'technology': '5G', 'traffic_type': 'Internet', 'start_time': '2026-07-14 18:08:11', 'duration_seconds': 2639, 'data_volume_mb': 575.02, 'event_time': '2026-07-15 13:07:10'}

RecordMetadata(topic='telecom_cdr_data', partition=0, topic_partition=TopicPartition(topic='telecom_cdr_data', partition=0), offset=31, timestamp=1784110030085, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=235, serialized_header_size=-1)

{'cdr_id': 2, 'subscription_id': 5, 'cell_id': 'CELL_000065', 'technology': '4G', 'traffic_type': 'Internet', 'start_time': '2026-07-14 22:15:02', 'duration_seconds': 550, 'data_volume_mb': 913.58, 'event_time': '2026-07-15 13:07:10'}

RecordMetadata(topic='telecom_cdr_data', partition=2, topic_partition=TopicPartition(topic='telecom_cdr_data', partition=2), offset=40, timestamp=1784110030202, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=234, serialized_header_size=-1)

root@kafka-broker:/# kafka-console-consumer \

> --bootstrap-server localhost:9092 \

> --topic telecom_cdr_data \

> --from-beginning \

> --max-messages 5

{"cdr_id": 2, "subscription_id": 408029, "cell_id": "CELL_000483", "technology": "5G", "traffic_type": "Internet", "start_time": "2026-07-15 11:56:03", "duration_seconds": 3460, "data_volume_mb": 1120.69, "event_time": "2026-07-15 12:07:47"}

{"cdr_id": 5, "subscription_id": 73775, "cell_id": "CELL_001130", "technology": "4G", "traffic_type": "Internet", "start_time": "2026-07-15 11:44:34", "duration_seconds": 2483, "data_volume_mb": 1746.32, "event_time": "2026-07-15 12:07:47"}

{"cdr_id": 6, "subscription_id": 357040, "cell_id": "CELL_001447", "technology": "4G", "traffic_type": "Internet", "start_time": "2026-07-15 11:23:39", "duration_seconds": 921, "data_volume_mb": 1943.43, "event_time": "2026-07-15 12:07:48"}

{"cdr_id": 7, "subscription_id": 445384, "cell_id": "CELL_000343", "technology": "4G", "traffic_type": "Internet", "start_time": "2026-07-15 12:04:59", "duration_seconds": 1265, "data_volume_mb": 2879.21, "event_time": "2026-07-15 12:07:48"}

{"cdr_id": 10, "subscription_id": 320186, "cell_id": "CELL_000192", "technology": "4G", "traffic_type": "Internet", "start_time": "2026-07-15 11:28:40", "duration_seconds": 349, "data_volume_mb": 1841.4, "event_time": "2026-07-15 12:07:48"}

Activat

Airflow > telecom-network-analytics > generators > streaming >  stream_network_performance.py >  main

```
1  import json
2  import time
3  import random
4
5  from datetime import datetime
6
7  from kafka import KafkaProducer
8
9  from generators.streaming.stream_utils import (
10      throughput_4g,
11      throughput_5g,
12      upload_speed,
13      latency,
14      packet_loss,
15      availability,
16      is_peak_hour
17  )
18
19  from generators.streaming.stream_distributions import (
20      weighted_choice,
21      TECH_DISTRIBUTION
22  )
23
24
25  TOPIC = "telecom_network_performance"
26
27
28  TOTAL_CELLS = 1500
29
30
```

```
32  # -----
33  # Active Users
34  # -----
35
36  def generate_active_users():
37
38      if is_peak_hour():
39
40          return random.randint(
41              3000,
42              8000
43          )
44
45
46      return random.randint(
47          500,
48          4000
49      )
50
51
52
53  # -----
54  # Network Condition
55  # -----
56
57  def network_condition(
58      availability_value,
59      packet_loss_value
60  ):
```

```
52
53 # -----
54 # Network Condition
55 # -----
56
57 def network_condition(
58     availability_value,
59     packet_loss_value
60 ):
61
62     if availability_value < 98:
63
64         return "Critical"
65
66
67     if packet_loss_value > 1:
68
69         return "Degraded"
70
71
72     return "Normal"
```

```
73
74
75
76
77 # -----
78 # Cell Generator
79 # -----
80
81 def generate_cell():
82
83     cell_id = (
84         f"CELL_{random.randint(1,TOTAL_CELLS):06d}"
85     )
86
87
88     technology = weighted_choice(
89         TECH_DISTRIBUTION
90     )
91
92
93     return cell_id, technology
```

```
98 # -----
99 # KPI Generator
100 # -----
101
102 def generate_kpi(measurement_id):
103
104     cell_id, technology = generate_cell()
105
106
107
108
109     download_mbps = (
110
111         throughput_4g()
112
113         if technology == "4G"
114
115         else throughput_5g()
116
117     )
118
119
120     upload = upload_speed(
121         technology
122     )
123
124
125     latency_value = latency(
126         technology
127     )
128
```

```
129
130     packet_loss_value = packet_loss()
131
132
133     availability_value = availability()
134
135
136
137     return {
138
139         "measurement_id":
140         |     measurement_id,
141
142
143         "cell_id":
144         |     cell_id,
145
146
147         "technology":
148         |     technology,
149
150
151         "download_mbps":
152         |     download_mbps,
153
154
155         "upload_mbps":
156         |     upload,
```

```
155
156     "upload_mbps":
157         |         upload,
158
159
160     "latency_ms":
161         |         latency_value,
162
163
164     "packet_loss":
165         |         packet_loss_value,
166
167
168     "availability":
169         |         availability_value,
170
171
172     "active_users":
173         |         generate_active_users(),
174
175
176     "network_condition":
177         |         network_condition(
178             |             availability_value,
179             |             packet_loss_value
180         |         ),
181
182
183     "event_time":
184         |         datetime.now().strftime(
185             |             "%Y-%m-%d %H:%M:%S"
```

```
176     "network_condition":
177         |         network_condition(
178             |             availability_value,
179             |             packet_loss_value
180         |         ),
181
182
183     "event_time":
184         |         datetime.now().strftime(
185             |             "%Y-%m-%d %H:%M:%S"
186         |         )
187
188 }
189
```

```
191
192
193 # -----
194 # Streaming
195 # -----
196
197 def main():
198
199
200
201     producer = KafkaProducer(
202
203         bootstrap_servers="localhost:9092",
204
205         value_serializer=lambda x:
206             json.dumps(x).encode("utf-8")
207
208     )
209
210
211     print(
212         "Network Performance streaming started"
213     )
214
215
216     counter = 0
217
218
219
220     while True:
```

```
219
220     while True:
221
222
223         counter += 1
224
225
226         data = generate_kpi(
227             counter
228         )
229
230
231         future = producer.send(
232             TOPIC,
233             data
234         )
235
236
237         try:
238
239
240             metadata = future.get(
241                 timeout=10
242             )
243
244
245             print(data)
246
247             print(metadata)
248
```

```

243
244
245         print(data)
246
247         print(metadata)
248
249
250
251     except Exception as e:
252
253
254         print(
255             "Kafka Error:",
256             e
257         )
258
259
260
261     time.sleep(
262         0.5
263     )
264

```

```

(.venv) PS F:\Airflow\telecom-network-analytics> python -m generators.streaming.stream_network_performance
Network Performance streaming started
{'measurement_id': 1, 'cell_id': 'CELL_000876', 'technology': '4G', 'download_mbps': 32.34, 'upload_mbps': 9.21, 'latency_ms': 38.49, 'packet_loss': 0.506, 'availability': 99.7, 'active_users': 2877, 'network_condition': 'Normal', 'event_time': '2026-07-15 13:01:30'}
RecordMetadata(topic='telecom_network_performance', partition=1, topic_partition=TopicPartition(topic='telecom_network_performance', partition=1), offset=9, timestamp=1784109690061, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=267, serialized_header_size=-1)
{'measurement_id': 2, 'cell_id': 'CELL_000849', 'technology': '4G', 'download_mbps': 43.36, 'upload_mbps': 12.22, 'latency_ms': 44.0, 'packet_loss': 0.373, 'availability': 99.72, 'active_users': 1612, 'network_condition': 'Normal', 'event_time': '2026-07-15 13:01:30'}
RecordMetadata(topic='telecom_network_performance', partition=1, topic_partition=TopicPartition(topic='telecom_network_performance', partition=1), offset=10, timestamp=1784109690569, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=268, serialized_header_size=-1)
{'measurement_id': 3, 'cell_id': 'CELL_000462', 'technology': '4G', 'download_mbps': 44.53, 'upload_mbps': 12.9, 'latency_ms': 34.52, 'packet_loss': 0.692, 'availability': 99.74, 'active_users': 3677, 'network_condition': 'Normal', 'event_time': '2026-07-15 13:01:31'}
RecordMetadata(topic='telecom_network_performance', partition=1, topic_partition=TopicPartition(topic='telecom_network_performance', partition=1), offset=11, timestamp=1784109691077, log_start_offset=0, checksum=None, serialized_key_size=-1, serialized_value_size=269, serialized_header_size=-1)

```

Data Ingestion Using NIFI

For batch

07:39:41 U

Canvas

Summary

Counter

Bulletin Board

Data Provenance

Controller Settings

Parameter Contexts

Flow Configuration History

Node Status History

System Diagnostics

Help

About

Appearance

Animations

Name ↑					
Type		Bundle	State		Scope
DBCPConnectionPool(PostgreSQL_Batch_DB)		org.apache.nifi - nifi-dbcp-serv...	Enabled		Controller

Controller Service Details

Settings

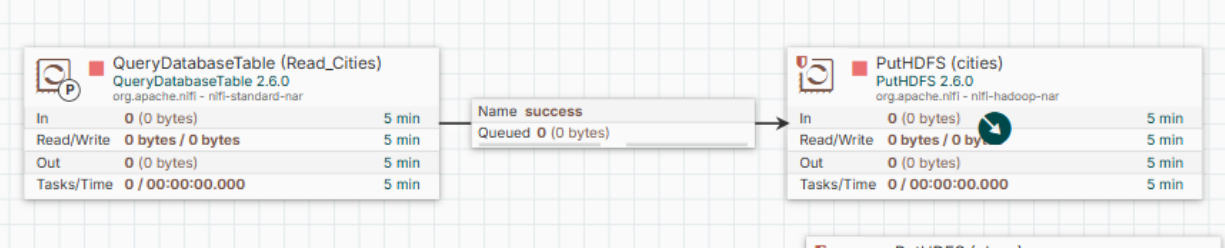
Properties

Required field

Property	Value
Database Connection URL	<i>i</i> jdbc:postgresql://external_postgres_...
Database Driver Class Name	<i>i</i> org.postgresql.Driver
Database Driver Location(s)	<i>i</i> /opt/nifi/nifi-current/lib/postgresql-4...
Kerberos User Service	<i>i</i> No value set
Database User	<i>i</i> external
Password	<i>i</i> Sensitive value set
Max Wait Time	<i>i</i> 500 millis
Max Total Connections	<i>i</i> 8

Required field

<i>EL</i> ✓ <i>PARAM</i> ⓧ	
Property	
Database Connection URL	<i>i</i> 1 jdbc:postgresql://external_postgres_db:5432/external
Database Driver Location(s)	1 /opt/nifi/nifi-current/lib/postgresql-42.7.12.jar
Kerberos User Service	
Database User	<i>i</i> external



Edit Processor | QueryDatabaseTable 2.6.0

Settings

Scheduling

Properties

Required field

Property	Value
Database Connection Pooling Ser...	DBCPCConnectionPool
Database Type	PostgreSQL
Table Name	network.cities

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

Required field



Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/batch/cities
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename

Edit Processor | PutHDFS 2.6.0

- Settings
- Scheduling
- Properties
- Relationships

Automatically Terminate/Retry Relationships ⓘ
failure

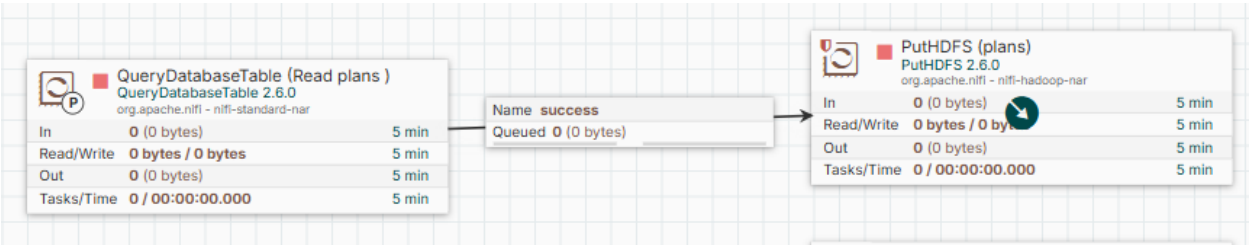
☒ terminate ☐ retry

Files that could not be written to HDFS for some reason are transferred to this relationship

success

☒ terminate ☐ retry

Files that have been successfully written to HDFS are transferred to this relationship



Edit Processor | QueryDatabaseTable 2.6.0

- Settings
- Scheduling
- Properties
- F

Required field +

Property	Value
Database Connection Pooling Ser... ⓘ	DBCPConnectionPool ⓘ
Database Type ⓘ	PostgreSQL ⓘ
Table Name ⓘ	network.plans ⓘ
Columns to Return ⓘ	No value set ⓘ
Additional WHERE clause ⓘ	No value set ⓘ
Custom Query ⓘ	No value set ⓘ
Maximum-value Columns ⓘ	No value set ⓘ

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

R

Required field +

Property	Value
Hadoop Configuration Resources	i /opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	i No value set ⋮
Additional Classpath Resources	i No value set
Directory	i /bronze/batch/plans
Conflict Resolution Strategy	i replace
Writing Strategy	i Write and rename

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Automatically Terminate/Retry Relationships i
failure

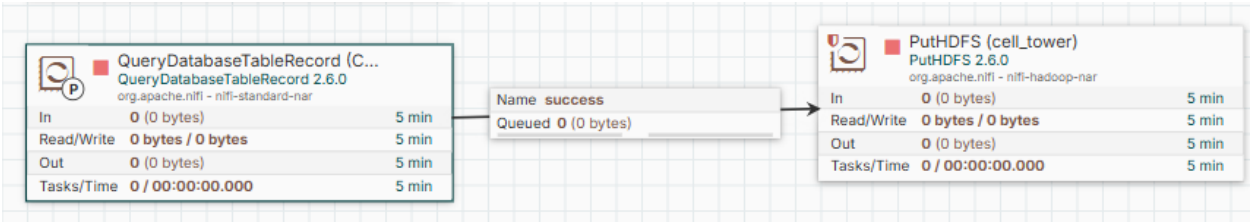
☒ terminate ☐ retry

Files that could not be written to HDFS for some reason are transferred to this relationship

success

☒ terminate ☐ retry

Files that have been successfully written to HDFS are transferred to this relationship



Required field



Property	Value
Database Connection Pooling Ser...	DBCPConnectionPool
Database Type	PostgreSQL
Table Name	network.cell_towers
Columns to Return	No value set
Additional WHERE clause	No value set
Custom Query	No value set
Record Writer	AvroRecordSetWriter(batch)

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

R

Required field



Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/batch/cell_towers
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename
Block Size	No value set

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Automatically Terminate/Retry Relationships ⓘ

failure

☒ terminate ☐ retry

Files that could not be written to HDFS for some reason are transferred to this relationship

success

☒ terminate ☐ retry

Files that have been successfully written to HDFS are transferred to this relationship

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

Required field +

Property

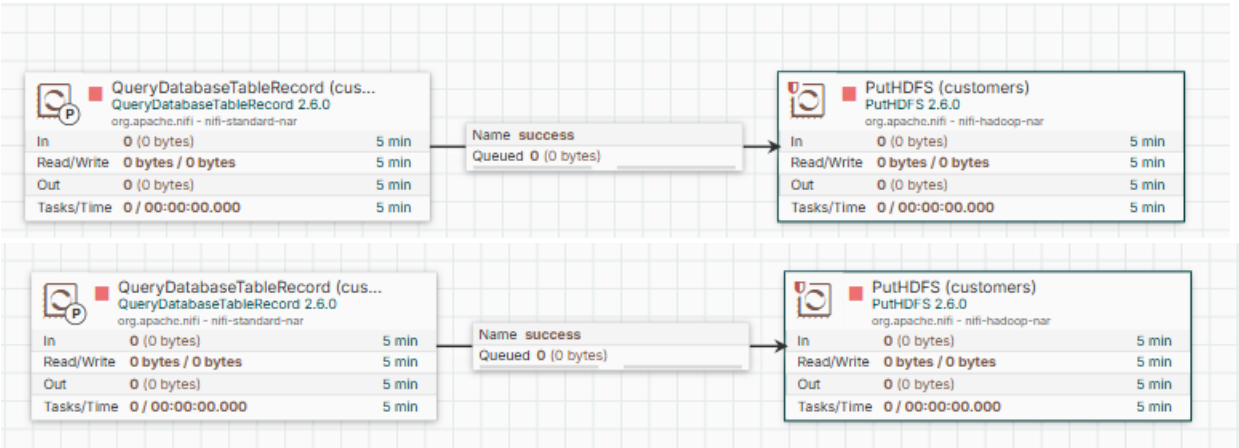
Hadoop Configuration Resources

Kerberos User Service

EL ✓

PARAM ✓

1 /opt/nifi/nifi-current/conf/core-site.xml,/opt/nifi/nifi-current/conf/hdfs-site.xml



Edit Processor | QueryDatabaseTableRecord 2.6.0

Settings

Scheduling

Properties

Required field



Property	Value
Database Connection Pooling Ser...	DBCPConnectionPool
Database Type	PostgreSQL
Table Name	network.customers
Columns to Return	No value set
Additional WHERE clause	No value set
Custom Query	No value set
Record Writer	AvroRecordSetWriter(batch)

Edit Processor | PutHDFS 2.6.0

Settings

Scheduling

Properties

Required field



Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/batch/customers
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename

Property

EL ✓ PARAM ✓

Hadoop Configuration Resources

Kerberos User Service

```
1 /opt/nifi/nifi-current/conf/core-site.xml,/opt/nifi/nifi-current/conf/hdfs-site.xml|
```

	QueryDatabaseTableRecord (su... QueryDatabaseTableRecord 2.6.0 org.apache.nifi - nifi-standard-nar
In	0 (0 bytes) 5 min
Read/Write	0 bytes / 0 bytes 5 min
Out	0 (0 bytes) 5 min
Tasks/Time	0 / 00:00:00.000 5 min

Name success
Queued 0 (0 bytes)

	PutHDFS (subscriptions) PutHDFS 2.6.0 org.apache.nifi - nifi-hadoop-nar
In	0 (0 bytes) 5 min
Read/Write	0 bytes / 0 bytes 5 min
Out	0 (0 bytes) 5 min
Tasks/Time	0 / 00:00:00.000 5 min

Edit Processor | QueryDatabaseTableRecord 2.6.0

Settings

Scheduling

Properties

Rel

Required field

+ \

Property	Value
Database Connection Pooling Ser...	 DBCPConnectionPool 
Database Type	 PostgreSQL
Table Name	 network.subscriptions
Columns to Return	 No value set
Additional WHERE clause	 No value set
Custom Query	 No value set
Record Writer	 AvroRecordSetWriter(batch) 

Edit Processor | PutHDFS 2.6.0

Settings

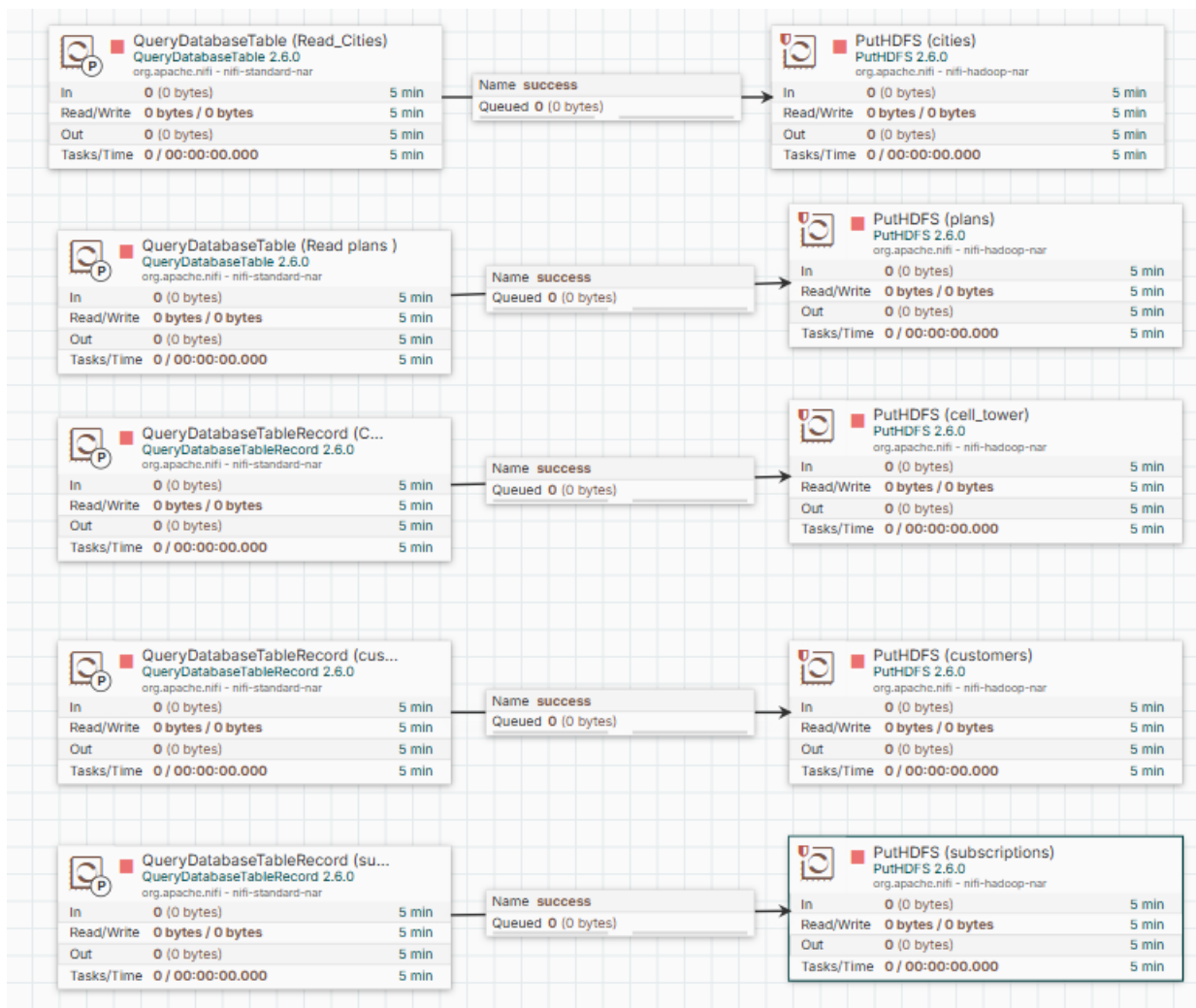
Scheduling

Properties

Required field

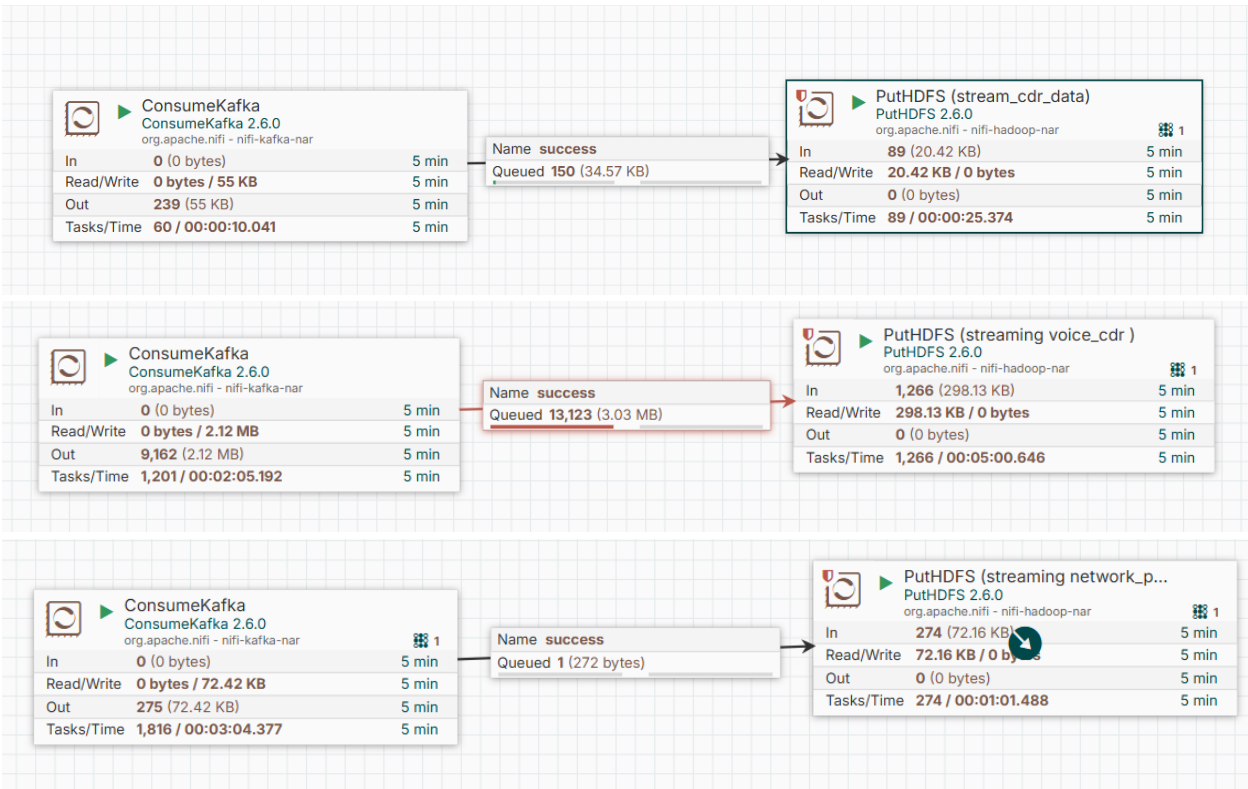


Property	Value
Hadoop Configuration Resources	 /opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	 No value set 
Additional Classpath Resources	 No value set
Directory	 /bronze/batch/subscriptions
Conflict Resolution Strategy	 replace
Writing Strategy	 Write and rename
Block Size	 No value set



Data Ingestion Using NIFI

For Stream Silver



Controller Services

NIFI Flow

Name ↑	Type	Bundle	State	Scope
AvroRecordSetWriter(batch)	AvroRecordSetWriter 2.6.0	org.apache.nifi - nifi-record-se...	Enabled	NIFI Flow
DBCPConnectionPool	DBCPConnectionPool 2.6.0	org.apache.nifi - nifi-dbcv-serv...	Enabled	NIFI Flow
DBCPConnectionPool	DBCPConnectionPool 2.6.0	org.apache.nifi - nifi-dbcv-serv...	Enabled	NIFI Flow
JsonTreeReader (batch)	JsonTreeReader 2.6.0	org.apache.nifi - nifi-record-se...	Enabled	NIFI Flow
Kafka3ConnectionService	Kafka3ConnectionService 2.6.0	org.apache.nifi - nifi-kafka-3-s...	Enabled	NIFI Flow

Controller Service Details

Kafka3ConnectionService 2.6.0

Settings

Properties

Comments

Required field

Verification

Property	Value
Bootstrap Servers	kafka-broker:29092
Security Protocol	PLAINTEXT
Transaction Isolation Level	Read Uncommitted
Max Poll Records	10000
Client Timeout	60 sec
Max Metadata Wait Time	5 sec
Acknowledgment Wait Time	5 sec

Edit Processor | ConsumeKafka 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

+ Verification



Property	Value
Kafka Connection Service	Kafka3ConnectionService
Group ID	nifi-test-123
Topic Format	names
Topics	telecom_cdr_data
Auto Offset Reset	earliest
Commit Offsets	true
Max Uncommitted Size	No value set

Click the button above to verify this component.

Processor Details | ConsumeKafka 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Verification

Property	Value
Kafka Connection Service	Kafka3ConnectionService
Group ID	nifi-voice-cdr
Topic Format	names
Topics	telecom_voice_cdr
Auto Offset Reset	earliest
Commit Offsets	true
Max Uncommitted Size	No value set
Max Uncommitted Time	100 millis

Processor Details | ConsumeKafka 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Property	Value
Kafka Connection Service	Kafka3ConnectionService
Group ID	nifi-network-performance
Topic Format	names
Topics	telecom_network_performance
Auto Offset Reset	earliest
Commit Offsets	true
Max Uncommitted Size	No value set
Max Uncommitted Time	100 millis

Verification

✓

Processor Details | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/stream/telecom_cdr_data
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename
Block Size	No value set
IO Buffer Size	No value set

Verification

✓

Processor Details | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Verification



Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/stream/telecom_voice_cdr
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename
Block Size	No value set
IO Buffer Size	No value set

Processor Details | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Verification

Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	No value set
Additional Classpath Resources	No value set
Directory	/bronze/stream/telecom_network_per...
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename
Block Size	No value set
IO Buffer Size	No value set

Property

Hadoop Configuration Resources

Kerberos User Service

Additional Classpath Resources

EL ✓ PARAM ✓

1 /opt/nifi/nifi-current/conf/core-site.xml,/opt/nifi/nifi-current/conf/hdfs-site.xml

Processor Details | PutHDFS 2.6.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

Property	Value
Hadoop Configuration Resources	 /opt/nifi/nifi-current/conf/core-site.x...
Kerberos User Service	 No value set
Additional Classpath Resources	 No value set
Directory	 /bronze/stream/telecom_network_per...
Conflict Resolution Strategy	 replace
Writing Strategy	 Write and rename
Block Size	 No value set
IO Buffer Size	 No value set

Verification

localhost:9021/clusters/2l9hfRW3T42fStGYSGU0iA/management/topics

Microsoft Teams test - Power BI كيف اتعلم لغة... (388) The O... مشاهدة فيلم... CV-Template | Tem... تعلم html بال... (441) Job Search 23_5_2023-202305... CV-Template | 1

confluent



cluster 1

CONTROLCENTER.CLUST...

Overview

Brokers

Topics

Connect

KSQL

Consumers

Replicators

All topics

☒ Hide internal topics

Topics Topic name	Partitions Total partitions
telecom_cdr_data	3
telecom_network_performance	3
telecom_voice_cdr	3

localhost:9021/clusters/2l9hfRW3T42fStGYSgU0iA/management/settings/brokers

Microsoft Teams test - Power BI كيف نتعلم لغة... (388) مشاهدة فيلم... CV-Template | Tem... تعلم html بال... (441) Job Search 23_5_2023-202305... CV-Templi

confluent

CONTROLCENTER.CLUST...

Overview

Brokers

Topics

Connect

KSQL

Consumers

Replicators

Cluster settings

Cluster settings

Kafka

Brokers

☒ Hide settings set to default value [Edit Settings](#) [Download](#)

GENERAL

broker.id	1
-----------	---

LISTENER

listeners	PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST...
advertised.listeners	PLAINTEXT://kafka-broker:29092,PLAINTEXT_...

LOG

log.dirs	/var/lib/kafka/data
----------	---------------------

Spark Transformation

For batch Silver

```
[3]: cities = spark.read.format("avro").load(
      "hdfs://namenode:9000/bronze/batch/cities"
    )

    cities.printSchema()
```

```
root
 |-- city_id: integer (nullable = true)
 |-- city_name: string (nullable = true)
 |-- region: string (nullable = true)
 |-- latitude: string (nullable = true)
 |-- longitude: string (nullable = true)
```

```
[4]: cities.show()
```

```
[Stage 0:>                                     (0 + 1) / 1]
+-----+-----+-----+-----+-----+
|city_id|city_name|region|latitude|longitude|
+-----+-----+-----+-----+-----+
| 1|Cairo|Greater Cairo|30.04440|31.23570|
| 2|Giza|Greater Cairo|30.01310|31.20890|
| 3|Alexandria|North Coast|31.20010|29.91870|
| 4|Mansoura|Delta|31.04090|31.37850|
| 5|Tanta|Delta|30.78650|31.00040|
| 6|Aswan|Upper Egypt|24.08890|32.89980|
| 7|Luxor|Upper Egypt|25.68720|32.63960|
| 8|Port Said|Canal|31.26530|32.30190|
| 9|Suez|Canal|29.96680|32.54980|
|10|Sharm El Sheikh|Sinai|27.91580|34.33000|
+-----+-----+-----+-----+-----+
```

Activate Wir
Go to Settings to

```
[5]: from pyspark.sql.functions import *
```

```
[6]: cities_silver = (
      cities
      .withColumn("latitude", col("latitude").cast("double"))
      .withColumn("longitude", col("longitude").cast("double"))
    )
```

```
[7]: cities_silver.printSchema()
```

```
root
 |-- city_id: integer (nullable = true)
 |-- city_name: string (nullable = true)
 |-- region: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
```

```
[8]: cities_silver.write \
      .mode("overwrite") \
      .parquet("hdfs://namenode:9000/silver/batch/cities")
```

```
[9]: plans = spark.read.format("avro").load(
      "hdfs://namenode:9000/bronze/batch/plans"
    )

    plans.printSchema()
```

```
root
 |-- plan_id: integer (nullable = true)
 |-- plan_name: string (nullable = true)
 |-- price: string (nullable = true)
 |-- internet_gb: integer (nullable = true)
 |-- voice_minutes: integer (nullable = true)
 |-- plan_type: string (nullable = true)
```

```
[10]: plans.show()
```

```
+-----+-----+-----+-----+-----+
|plan_id|plan_name|price|internet_gb|voice_minutes|plan_type|
+-----+-----+-----+-----+-----+
| 1|Basic|75.00|6|600|CONSUMER|
| 2|Medium|150.00|18|1600|CONSUMER|
| 3|Premium|400.00|55|4000|CONSUMER|
| 4|Corporate VIP|850.00|140|10000|ENTERPRISE|
+-----+-----+-----+-----+-----+
```

Activate Wir
Go to Settings to

```
[11]: plans_silver = (  
      plans  
      .withColumn('price', col('price').cast('float'))  
    )
```

```
[12]: plans_silver.printSchema()  
  
root  
|-- plan_id: integer (nullable = true)  
|-- plan_name: string (nullable = true)  
|-- price: float (nullable = true)  
|-- internet_gb: integer (nullable = true)  
|-- voice_minutes: integer (nullable = true)  
|-- plan_type: string (nullable = true)
```

```
[14]: plans_silver.write\  
      .mode('overwrite')\  
      .format('parquet')\  
      .save("hdfs://namenode:9000/silver/batch/plans")
```

```
[15]: customers = spark.read.format("avro").load(  
      "hdfs://namenode:9000/bronze/batch/customers"  
    )  
  
customers.printSchema()  
  
root  
|-- customer_id: long (nullable = true)  
|-- gender: string (nullable = true)  
|-- birth_date: string (nullable = true)  
|-- segment: string (nullable = true)  
|-- city_id: integer (nullable = true)
```

```
[16]: customers.show()
```

customer_id	gender	birth_date	segment	city_id
1	Male	1085443200000	Individual	6
2	Female	1925856000000	Individual	4
3	Female	2062368000000	Business	6
4	Female	6796224000000	Business	2
5	Male	1078012800000	Individual	6
6	Female	6880032000000	Individual	1
7	Male	6908544000000	Individual	1
8	Female	2160000000000	Individual	1
9	Male	6840288000000	Individual	2
10	Male	2101248000000	Individual	1
11	Female	6652800000000	Individual	6
12	Female	6935328000000	Individual	1
13	Male	1082764800000	Business	5
14	Male	6730560000000	Individual	5
15	Male	1077494400000	Individual	5
16	Female	2024352000000	Individual	2
17	Male	2115936000000	Individual	3
18	Male	1952640000000	Individual	1
19	Male	6858432000000	Individual	6
20	Male	6689088000000	Business	5

only showing top 20 rows

```
[17]: customers_silver = (  
    customers  
    .withColumn(  
        "birth_date",  
        from_unixtime(col("birth_date").cast("long") / 1000).cast("date")  
    )  
)
```

```
[18]: customers_silver.printSchema()
```

```
root  
|-- customer_id: long (nullable = true)  
|-- gender: string (nullable = true)  
|-- birth_date: date (nullable = true)  
|-- segment: string (nullable = true)  
|-- city_id: integer (nullable = true)
```

```
[20]: customers_silver.write\  
      .mode('overwrite')\  
      .format('parquet')\  
      .save("hdfs://namenode:9000/silver/batch/customers")
```

```
[21]: customers_check = spark.read.parquet(  
      "hdfs://namenode:9000/silver/batch/customers"  
      )  
  
customers_check.printSchema()  
customers_check.show(5)
```

```
root  
|-- customer_id: long (nullable = true)  
|-- gender: string (nullable = true)  
|-- birth_date: date (nullable = true)  
|-- segment: string (nullable = true)  
|-- city_id: integer (nullable = true)  
  
+-----+-----+-----+-----+-----+  
|customer_id|gender|birth_date|  segment|city_id|  
+-----+-----+-----+-----+-----+  
|          1| Male|2004-05-25|Individual|      6|  
|          2|Female|1976-02-08|Individual|      4|  
|          3|Female|1976-07-15| Business|      6|  
|          4|Female|1991-07-16| Business|      2|  
|          5|  Male|2004-02-29|Individual|      6|  
+-----+-----+-----+-----+-----+  
only showing top 5 rows
```

```
[22]: subscriptions = spark.read.format("avro").load(  
      "hdfs://namenode:9000/bronze/batch/subscriptions"  
      )  
  
subscriptions.printSchema()
```

```
root  
|-- subscription_id: long (nullable = true)  
|-- customer_id: long (nullable = true)  
|-- plan_id: integer (nullable = true)  
|-- msisdn: string (nullable = true)  
|-- activation_date: string (nullable = true)  
|-- status: string (nullable = true)  
|-- subscription_type: string (nullable = true)
```

```
[23]: subscriptions.show()
```

subscription_id	customer_id	plan_id	msisdn	activation_date	status	subscription_type
1	241364	1	+2001595442661	1714780800000	Active	CONSUMER
2	143663	2	+2001152815141	1643760000000	Active	CONSUMER
3	842668	4	+2001099466896	1743292800000	Active	ENTERPRISE
4	780552	2	+2001293780172	1725148800000	Active	CONSUMER
5	55146	1	+2001530371970	1623369600000	Active	CONSUMER
6	255893	1	+2001065681311	1675641600000	Active	CONSUMER
7	571054	4	+2001245758073	1642809600000	Active	ENTERPRISE
8	672143	1	+2001110227101	1674864000000	Active	CONSUMER
9	667386	1	+2001535295171	1751155200000	Active	CONSUMER
10	964765	4	+2001528070961	1634515200000	Active	ENTERPRISE
11	855857	4	+2001247721140	1686009600000	Active	ENTERPRISE
12	897188	2	+2001096183745	1615593600000	Active	CONSUMER
13	620347	2	+2001117293455	1775692800000	Active	CONSUMER
14	27003	1	+2001085347234	1720224000000	Active	CONSUMER
15	255569	2	+2001114264379	1711756800000	Terminated	CONSUMER
16	588750	4	+2001562300487	1690934400000	Active	ENTERPRISE
17	982406	3	+2001514630883	1693785600000	Active	CONSUMER
18	244848	1	+2001075981228	1733702400000	Active	CONSUMER
19	36081	4	+2001132358254	1713744000000	Suspended	ENTERPRISE
20	439271	1	+2001597870310	1734480000000	Suspended	CONSUMER

only showing top 20 rows

```
[24]: subscriptions_silver = (  
    subscriptions  
    .withColumn(  
        "activation_date",  
        from_unixtime(  
            col("activation_date").cast("long") / 1000  
        ).cast("date")  
    )  
)
```

```
[25]: subscriptions_silver.printSchema()
```

```
root  
|-- subscription_id: long (nullable = true)  
|-- customer_id: long (nullable = true)  
|-- plan_id: integer (nullable = true)  
|-- msisdn: string (nullable = true)  
|-- activation_date: date (nullable = true)  
|-- status: string (nullable = true)  
|-- subscription_type: string (nullable = true)
```

[26]: `subscriptions_silver.show()`

subscription_id	customer_id	plan_id	msisdn	activation_date	status	subscription_type
1	241364	1	+2001595442661	2024-05-04	Active	CONSUMER
2	143663	2	+2001152815141	2022-02-02	Active	CONSUMER
3	842668	4	+2001099466896	2025-03-30	Active	ENTERPRISE
4	780552	2	+2001293780172	2024-09-01	Active	CONSUMER
5	55146	1	+2001530371970	2021-06-11	Active	CONSUMER
6	255893	1	+2001065681311	2023-02-06	Active	CONSUMER
7	571054	4	+2001245758073	2022-01-22	Active	ENTERPRISE
8	672143	1	+2001110227101	2023-01-28	Active	CONSUMER
9	667386	1	+2001535295171	2025-06-29	Active	CONSUMER
10	964765	4	+2001528070961	2021-10-18	Active	ENTERPRISE
11	855857	4	+2001247721140	2023-06-06	Active	ENTERPRISE
12	897188	2	+2001096183745	2021-03-13	Active	CONSUMER
13	620347	2	+2001117293455	2026-04-09	Active	CONSUMER
14	27003	1	+2001085347234	2024-07-06	Active	CONSUMER
15	255569	2	+2001114264379	2024-03-30	Terminated	CONSUMER
16	588750	4	+2001562300487	2023-08-02	Active	ENTERPRISE
17	982406	3	+2001514630883	2023-09-04	Active	CONSUMER
18	244848	1	+2001075981228	2024-12-09	Active	CONSUMER
19	36081	4	+2001132358254	2024-04-22	Suspended	ENTERPRISE
20	439271	1	+2001597870310	2024-12-18	Suspended	CONSUMER

only showing top 20 rows

```
[27]: subscriptions_silver = (  
    subscriptions_silver  
    .withColumn("country_code", lit("+20"))  
    .withColumn(  
        "msisdn",  
        regexp_replace(col("msisdn"), r"^\+20", "")  
    )  
)
```

```
[28]: subscriptions_silver.select(  
    "msisdn",  
    "country_code"  
)>.show(10)
```

```
+-----+-----+  
|    msisdn|country_code|  
+-----+-----+  
|01595442661|      +20|  
|01152815141|      +20|  
|01099466896|      +20|  
|01293780172|      +20|  
|01530371970|      +20|  
|01065681311|      +20|  
|01245758073|      +20|  
|01110227101|      +20|  
|01535295171|      +20|  
|01528070961|      +20|  
+-----+-----+  
only showing top 10 rows
```

```
[29]: subscriptions_silver.write\  
      .mode('overwrite')\  
      .format('parquet')\  
      .save("hdfs://namenode:9000/silver/batch/subscriptions")
```

```
[30]: cell_towers = spark.read.format('avro').load(  
      "hdfs://namenode:9000/bronze/batch/cell_towers"  
  
      )  
  
      cell_towers.printSchema()
```

```
root  
 |-- cell_id: string (nullable = true)  
 |-- technology: string (nullable = true)  
 |-- latitude: string (nullable = true)  
 |-- longitude: string (nullable = true)  
 |-- city_id: integer (nullable = true)
```

```
[31]: cell_towers.show()
```

```
+-----+-----+-----+-----+-----+  
|   cell_id|technology| latitude|longitude|city_id|  
+-----+-----+-----+-----+-----+  
|CELL_000001|      4G|27.949815|34.330592|    10|  
|CELL_000002|      4G|29.942648|31.142342|     2|  
|CELL_000003|      4G|31.227632|32.306349|     8|  
|CELL_000004|      4G|30.085837|31.209520|     1|  
|CELL_000005|      5G|30.008061|31.281204|     1|  
|CELL_000006|      4G|31.007659|31.445560|     4|  
|CELL_000007|      4G|30.005112|31.189311|     2|  
|CELL_000008|      4G|30.865030|30.950781|     5|  
|CELL_000009|      4G|29.971490|31.259248|     1|  
|CELL_000010|      5G|31.275720|29.887946|     3|  
|CELL_000011|      4G|29.977193|31.217263|     2|  
|CELL_000012|      4G|31.096413|31.305275|     4|  
|CELL_000013|      4G|30.021417|31.257719|     1|  
|CELL_000014|      4G|29.984134|31.187550|     1|  
|CELL_000015|      5G|30.076382|31.159878|     1|  
|CELL_000016|      5G|30.039867|31.154679|     2|  
|CELL_000017|      4G|30.745436|31.071300|     5|  
|CELL_000018|      4G|30.767464|31.003993|     5|  
|CELL_000019|      5G|29.991497|31.268354|     2|  
|CELL_000020|      4G|31.033350|31.458323|     4|  
+-----+-----+-----+-----+-----+
```

only showing top 20 rows

```
[32]: cell_towers_silver=(
      cell_towers
      .withColumn('latitude',col('latitude').cast('double'))
      .withColumn('longitude',col('longitude').cast('double'))
      )
```

```
[33]: cell_towers_silver.printSchema()
```

```
root
 |-- cell_id: string (nullable = true)
 |-- technology: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- city_id: integer (nullable = true)
```

```
[34]: cell_towers_silver.show()
```

```
+-----+-----+-----+-----+-----+
|  cell_id|technology| latitude|longitude|city_id|
+-----+-----+-----+-----+-----+
|CELL_000001|      4G|27.949815|34.330592|    10|
|CELL_000002|      4G|29.942648|31.142342|     2|
|CELL_000003|      4G|31.227632|32.306349|     8|
|CELL_000004|      4G|30.085837| 31.20952|     1|
|CELL_000005|      5G|30.008061|31.281204|     1|
|CELL_000006|      4G|31.007659| 31.44556|     4|
|CELL_000007|      4G|30.005112|31.189311|     2|
|CELL_000008|      4G| 30.86503|30.950781|     5|
|CELL_000009|      4G| 29.97149|31.259248|     1|
|CELL_000010|      5G| 31.27572|29.887946|     3|
|CELL_000011|      4G|29.977193|31.217263|     2|
|CELL_000012|      4G|31.096413|31.305275|     4|
|CELL_000013|      4G|30.021417|31.257719|     1|
|CELL_000014|      4G|29.984134| 31.18755|     1|
|CELL_000015|      5G|30.076382|31.159878|     1|
|CELL_000016|      5G|30.039867|31.154679|     2|
|CELL_000017|      4G|30.745436| 31.0713|     5|
|CELL_000018|      4G|30.767464|31.003993|     5|
|CELL_000019|      5G|29.991497|31.268354|     2|
|CELL_000020|      4G| 31.03335|31.458323|     4|
+-----+-----+-----+-----+-----+
only showing top 20 rows
```

Spark Transformation

For Streaming Silver

```
[37]: voice_raw = spark.readStream \
      .format("kafka") \
      .option("kafka.bootstrap.servers", "kafka-broker:29092") \
      .option("subscribe", "telecom_voice_cdr") \
      .option("startingOffsets", "latest") \
      .load()
```

```
[38]: query = (
      voice_raw
      .writeStream
      .format("console")
      .outputMode("append")
      .option("truncate", False)
      .start()
    )
```

```
-----+-----+
[null][7B 22 63 64 72 5F 69 64 22 3A 20 39 36 32 2C 20 22 73 75 62 73 63 72 69 70 74 69 6F 6E 5F 69 64 22 3A 20 33 2C 20 22 73 74 61 72 74 5F 74 69 6
D 65 22 3A 20 22 32 30 32 36 2D 30 38 2D 31 39 20 30 35 3A 30 32 3A 34 35 22 2C 20 22 64 75 72 61 74 69 6F 6E 5F 73 65 63 6F 6E 64 73 22 3A 20 32 34
37 2C 20 22 63 65 6C 6C 5F 69 64 22 3A 20 22 43 45 4C 4C 5F 30 30 30 33 37 30 22 2C 20 22 74 65 63 68 6E 6F 6C 6F 67 79 22 3A 20 22 34 47 22 2C 20 22
63 61 6C 6C 5F 73 74 61 74 75 73 22 3A 20 22 43 6F 6D 70 6C 65 74 65 64 22 2C 20 22 74 65 72 6D 69 6E 61 74 69 6F 6E 5F 63 61 75 73 65 22 3A 20 22 4E
4F 52 4D 41 4C 22 2C 20 22 65 76 65 6E 74 5F 74 69 6D 65 22 3A 20 22 32 30 32 36 2D 30 38 2D 31 39 20 30 35 3A 35 31 3A 34 30 22 7D] |telecom_voice
_cdr|2 |26625 |2026-08-19 02:51:40.354|0 |
[null][7B 22 63 64 72 5F 69 64 22 3A 20 39 36 33 2C 20 22 73 75 62 73 63 72 69 70 74 69 6F 6E 5F 69 64 22 3A 20 32 2C 20 22 73 74 61 72 74 5F 74 69 6
D 65 22 3A 20 22 32 30 32 36 2D 30 38 2D 31 39 20 30 35 3A 32 32 3A 34 33 22 2C 20 22 64 75 72 61 74 69 6F 6E 5F 73 65 63 6F 6E 64 73 22 3A 20 31 32
30 2C 20 22 63 65 6C 6C 5F 69 64 22 3A 20 22 43 45 4C 4C 5F 30 30 30 30 36 22 2C 20 22 74 65 63 68 6E 6F 6C 6F 67 79 22 3A 20 22 34 47 22 2C 20 22
63 61 6C 6C 5F 73 74 61 74 75 73 22 3A 20 22 43 6F 6D 70 6C 65 74 65 64 22 2C 20 22 74 65 72 6D 69 6E 61 74 69 6F 6E 5F 63 61 75 73 65 22 3A 20 22 4E
4F 52 4D 41 4C 22 2C 20 22 65 76 65 6E 74 5F 74 69 6D 65 22 3A 20 22 32 30 32 36 2D 30 38 2D 31 39 20 30 35 3A 35 31 3A 34 30 22 7D] |telecom_voice
_cdr|2 |26625 |2026-08-19 02:51:40.354|0 |
```

```
[40]: from pyspark.sql.types import (
      StructType,
      StructField,
      IntegerType,
      StringType
    )

    voice_schema = StructType([
      StructField("cdr_id", IntegerType(), True),
      StructField("subscription_id", IntegerType(), True),
      StructField("start_time", StringType(), True),
      StructField("duration_seconds", IntegerType(), True),
      StructField("cell_id", StringType(), True),
      StructField("technology", StringType(), True),
      StructField("call_status", StringType(), True),
      StructField("termination_cause", StringType(), True),
      StructField("event_time", StringType(), True)
    ])

    G, call_status = Completed, termination_cause = NORMAL, event_time = "2026-08-19 05:56:32"} |telecom_voice_cdr|0 |27122 |2026-08-
19 02:56:32.244|
[null]{"cdr_id": 186, "subscription_id": 2, "start_time": "2026-08-19 05:29:26", "duration_seconds": 197, "cell_id": "CELL_000206", "technology": "4
G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:32"} |telecom_voice_cdr|0 |27123 |2026-08-
19 02:56:32.314|
[null]{"cdr_id": 188, "subscription_id": 4, "start_time": "2026-08-19 05:55:58", "duration_seconds": 74, "cell_id": "CELL_000472", "technology": "4
G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:32"} |telecom_voice_cdr|0 |27124 |2026-08-
19 02:56:32.36 |
[null]{"cdr_id": 145, "subscription_id": 3, "start_time": "2026-08-19 05:14:56", "duration_seconds": 103, "cell_id": "CELL_000903", "technology": "4
G", "call_status": "Failed", "termination_cause": "NETWORK_FAILURE", "event_time": "2026-08-19 05:56:31"} |telecom_voice_cdr|1 |27246 |2026-08-
19 02:56:31.379|
```

```
voice_parsed = (
    voice_raw
    .select(
        from_json(
            col("value").cast("string"),
            voice_schema
        ).alias("data")
    )
    .select("data.*")
)
```

[null,{"cdr_id": 419, "subscription_id": 2, "start_time": "2026-08-19 05:47:09", "duration_seconds": 67, "cell_id": "CELL_000918", "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:37"}]	telecom_voice_cdr 1	27348	2026-08-19 02:56:37.792]
[null,{"cdr_id": 423, "subscription_id": 3, "start_time": "2026-08-19 05:18:43", "duration_seconds": 79, "cell_id": "CELL_000492", "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:37"}]	telecom_voice_cdr 1	27349	2026-08-19 02:56:37.884]
[null,{"cdr_id": 424, "subscription_id": 4, "start_time": "2026-08-19 05:29:41", "duration_seconds": 156, "cell_id": "CELL_000614", "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:37"}]	telecom_voice_cdr 1	27350	2026-08-19 02:56:37.907]
[null,{"cdr_id": 426, "subscription_id": 1, "start_time": "2026-08-19 05:43:36", "duration_seconds": 238, "cell_id": "CELL_000022", "technology": "4G", "call_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:37"}]	telecom_voice_cdr 1	27351	2026-08-19 02:56:37.951]

```
[42]: query = (
    voice_parsed
    .writeStream
    .format("console")
    .outputMode("append")
    .option("truncate", False)
    .start()
)
```

2:56:43.39]	[{"cdr_status": "Dropped", "termination_cause": "DROP", "event_time": "2026-08-19 05:56:43"}]	telecom_voice_cdr 1	27428	2026-08-19 0	
[null{"cdr_id": 649, "subscription_id": 1, "start_time": "2026-08-19 05:26:26", "duration_seconds": 117, "cell_id": "CELL_000409", "technology": "4	G", "cdr_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:43"}]	telecom_voice_cdr 1	27429	2026-08-19 0	
2:56:43.437]	[{"cdr_id": 650, "subscription_id": 5, "start_time": "2026-08-19 05:55:27", "duration_seconds": 117, "cell_id": "CELL_000975", "technology": "4	G", "cdr_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:43"}]	telecom_voice_cdr 1	27430	2026-08-19 0
2:56:43.461]	[{"cdr_id": 651, "subscription_id": 4, "start_time": "2026-08-19 05:35:44", "duration_seconds": 87, "cell_id": "CELL_000834", "technology": "4	G", "cdr_status": "Completed", "termination_cause": "NORMAL", "event_time": "2026-08-19 05:56:43"}]	telecom_voice_cdr 1	27431	2026-08-19 0
2:56:43.484]					

```
[47]: from pyspark.sql.functions import col, from_json, to_timestamp
```

```
voice_silver = (  
    voice_raw  
    .select(  
        col("key").cast("string").alias("kafka_key"),  
        col("value").cast("string").alias("json_value"),  
        col("topic"),  
        col("partition"),  
        col("offset"),  
        col("timestamp").alias("kafka_timestamp")  
    )  
    .withColumn(  
        "data",  
        from_json(col("json_value"), voice_schema)  
    )  
    .select(  
        "kafka_key",  
        "data.*",  
        "topic",  
        "partition",  
        "offset",  
        "kafka_timestamp"  
    )  
)
```

```
[53]: voice_silver = (  
    voice_silver  
    .withColumn(  
        "start_time",  
        to_timestamp("start_time", "yyyy-MM-dd HH:mm:ss")  
    )  
    .withColumn(  
        "event_time",  
        to_timestamp("event_time", "yyyy-MM-dd HH:mm:ss")  
    )  
  
    .withColumn("event_date", to_date("event_time"))  
    .withColumn("event_hour", hour("event_time"))  
)
```

```
[54]: voice_silver = (
    voice_silver
    .withColumn(
        "is_valid",
        (
            col("cdr_id").isNotNull()
            & col("subscription_id").isNotNull()
            & col("start_time").isNotNull()
            & col("duration_seconds").isNotNull()
            & (col("duration_seconds") >= 0)
            & col("cell_id").isNotNull()
            & col("technology").isNotNull()
            & col("call_status").isNotNull()
            & col("event_time").isNotNull()
        )
    )
)
```

```
[50]: query = (
    voice_silver
    .writeStream
    .format("parquet")
    .outputMode("append")
    .option("path", "/data/silver/telecom_voice_cdr")
    .option("checkpointLocation", "/data/checkpoints/silver/telecom_voice_cdr")
    .start()
)
```

```
-----
|cdr_id|subscription_id|start_time      |duration_seconds|cell_id  |technology|call_status|termination_cause|event_time      |
-----+-----+-----+-----+-----+-----+-----+-----+-----+
|13    |2              |2026-08-19 06:02:13|266            |CELL_000043|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
|15    |2              |2026-08-19 05:32:33|109            |CELL_000216|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
|17    |3              |2026-08-19 05:37:32|76             |CELL_000524|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
|22    |4              |2026-08-19 05:23:05|117            |CELL_001446|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
|24    |4              |2026-08-19 06:01:30|67             |CELL_001260|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
|26    |1              |2026-08-19 06:00:00|67             |CELL_000000|4G       |Completed |NORMAL           |2026-08-19 06:08:33|
```

Activate Wi

```
[*]: query.awaitTermination()
```

```
[55]: voice_silver.printSchema()
```

```
root
|-- kafka_key: string (nullable = true)
|-- cdr_id: integer (nullable = true)
|-- subscription_id: integer (nullable = true)
|-- start_time: timestamp (nullable = true)
|-- duration_seconds: integer (nullable = true)
|-- cell_id: string (nullable = true)
|-- technology: string (nullable = true)
|-- call_status: string (nullable = true)
|-- termination_cause: string (nullable = true)
|-- event_time: timestamp (nullable = true)
|-- topic: string (nullable = true)
|-- partition: integer (nullable = true)
|-- offset: long (nullable = true)
|-- kafka_timestamp: timestamp (nullable = true)
|-- is_valid: boolean (nullable = true)
|-- event_date: date (nullable = true)
|-- event_hour: integer (nullable = true)
```

```
[66]: silver_stream = (
      voice_silver
      .filter("is_valid = true")
    )
```

Batch: 122

cdr_id	subscription_id	start_time	duration_seconds	cell_id	technology	call_status	termination_cause	event_time
2	2	2026-08-19 06:31:07	184	CELL_000463	4G	Completed	NORMAL	2026-08-19 06:34:05
3	5	2026-08-19 05:52:49	105	CELL_000844	4G	Completed	NORMAL	2026-08-19 06:34:06
5	3	2026-08-19 05:38:23	227	CELL_000545	4G	Completed	NORMAL	2026-08-19 06:34:06
6	2	2026-08-19 06:03:10	241	CELL_001429	4G	Completed	NORMAL	2026-08-19 06:34:06
8	3	2026-08-19 06:05:54	77	CELL_001401	4G	Completed	NORMAL	2026-08-19 06:34:06
9	3	2026-08-19 06:08:29	107	CELL_000591	4G	Completed	NORMAL	2026-08-19 06:34:06